Functions

There is a set of functions in PHP that see the most use in logical expressions. These functions typically test variables for their type or even if they exist at all. Knowing in advance that a variable exists, has a certain type, or contains non-null or non-false data means you can fine-tune the code that comes next. These functions are type-strict, in that the variable being tested must be of the type concerned and not coercible to that type or capable of behaving like that type (`is_array(new ArrayObject())` would return `false`, for example). The following functions all return a `bool` value:

Table 4-2. *Control functions*

`empty($var)`	Problematic, see explanation below
`is_array($var)`	`$var` is type `array` (not array-like)
`is_bool($var)`	`$var` is type `bool`
`is_callable($var)`	`$var` can be invoked
`is_countable($var)`	`$var` is an `array` or implements `Countable`

`is_double($var)`
`is_float($var)`
`is_real($var)`
	`$var` is type `float`

`is_int($var)`
`is_integer($var)`
`is_long($var)`
	`$var` is type `int`

`is_iterable($var)`	`$var` is `array`|`Traversable` or (PHP 8.2) `iterable` psuedo-type
`is_null($var)`	`$var` has the `null` value
`is_numeric($var)`	`$var` is either `int`, `float`, or numeric `string`
`is_object($var)`	`$var` is an object instance
`is_resource($var)`	`$var` is a special reference to external resource (e.g., file descriptor)
`is_scalar($var)`	`$var` is `int`, `float`, `string`, or `bool`
`is_string($var)`	`$var` is `string`
`isset($var)`	`$var` is declared, and has non-null value